

Pollen forecast project

Léon Mercier

13.9.2026

What is this?

An ETL pipeline that downloads a pollen allergy forecast, builds charts from it and publishes them on the web

Check it out

Repo: <https://github.com/LeonMercier/pollen-sh>

Site: <http://www.pollencast.eu/>

- Learn:
 - Data engineering concepts
 - Python
 - Self-hosting and containers
 - Infrastructure as code
- Create a resource for pollen allergy sufferers to manage their symptoms
 - Easily see what days and times of day symptoms are likely to happen

What already exists

norkko.fi

- Lack of details:
 - Pollen level: scale from 0 to 3
 - Forecast: three days, no hourly data
 - Only a few locations
- In reality pollen levels vary wildly depending on time of day and weather
- <https://norkko.fi/>

EU Copernicus website

- Has animated map and charts
- Hard to discover and navigate
- Not user friendly
- https://atmosphere.copernicus.eu/charts/packages/cams_air_quality/

The data source

- EU Copernicus: European Union's Earth Observation Programme
- Releases daily, guaranteed complete at 10:00 UTC
- 96 hours length, 1 hour granularity
- All of continental Europe and Iceland
- 0.1 degree grid (about 10 km x 10 km grid cells)
- Six pollen types

From Azure to a single VPS

The first version ran on Azure: Data Factory orchestrating pySpark notebooks on Databricks, PostgreSQL Flexible Server, App Service, Blob Storage, all built by Terraform.

It worked, but for a low-traffic portfolio site it was a lot of moving parts — and a lot of monthly cost.

The rewrite targets one VPS. Same data, same charts, four containers.

The new stack

Layer	Before	After
Orchestration	Azure Data Factory	systemd timer
Processing	pySpark on Databricks	plain Python + numpy
Database	Azure PostgreSQL	PostgreSQL 17 in a container
Backend	Azure App Service	FastAPI + uvicorn
Frontend	Azure Blob static site	Caddy serving static files
TLS	—	Caddy, automatic
IaC	Terraform	Compose + a systemd unit

The key insight: the data is small

The old pipeline had elaborate batching — accumulate five GRIB messages, convert to Spark, write parquet, call the garbage collector, repeat.

That existed to work around Databricks JVM memory behaviour, not because the data is big:

- Bounding box: $171 \times 214 = \mathbf{36,594}$ grid cells
- 6 species $\times$ 97 hourly leadtimes = **582 GRIB messages**
- Whole forecast as float32: **81 MB**

It fits in memory. So: read the GRIB straight into one numpy array, load it in a single pass. No batching, no silver layer, no Spark.

Shaping the table around the query

The API only ever asks one thing: *every hourly value at one grid cell*.

Long format — one row per (cell, species, hour) — answers that with 582 rows and costs 21.3 M rows per day.

Instead, store the whole 97-hour series for a (cell, species) pair in one `real[]` column:

	Long format	Array format
Rows per run	21,300,000	219,564
On disk	~2.6 GB	104 MB
Load time	3–5 min	10 s
Rows per API query	582	6

A denormalised serving layer, shaped to the access pattern.

Geocoding without a spatial database

Each city has to be matched to its nearest forecast grid cell.

The old version: Spark cross join of 5,450 cities against 36,594 grid points — 199 million pairs — then a window function ranking by Haversine distance.

But the grid is *regular*. So the nearest cell is just:

```
lat_idx = round((lat - lat0) / step)
lon_idx = round((lon - lon0) / step)
```

Locally, great-circle distance is near-monotone in a separable quantity, so rounding each axis independently minimises it.

Does that actually hold?

Checked by brute force against real Haversine, over every city and every cell:

Cities snapped to a genuinely farther cell	10 of 5,450
Worst excess distance	1.7 m
Mean city-to-cell distance	3.39 km
Grid cell size	~11 km

The disagreements are all cities sitting within about 2 metres of exactly equidistant between two neighbours.

199 million distance calculations replaced by two `round()` calls — and no PostGIS.

Zero-downtime loads

The daily load still uses a blue-green swap:

- 1 COPY into a staging table
- 2 Build the primary key there
- 3 Validate: exact row count, all 6 species, every array 97 long
- 4 Rename staging into place, in one transaction, together with the run metadata
- 5 Drop the old table

A failed run changes nothing — yesterday's forecast simply stays live. The API can never read a start time that disagrees with the data it indexes into.

Not hardcoding what you can discover

The grid origin and spacing are read out of the GRIB file on every run and written to a grid table.

Nothing downstream assumes where the CAMS grid points sit. If Copernicus changes the grid, the pipeline fails loudly instead of silently mis-snapping every city in the database.

Same idea for the pollen species: olive arrives as the bare numeric code 64002, which is normalised at the boundary so no magic number reaches the database.

Results

Full daily run	110 s (99 s of that is the download)
Database load	10 s
Whole database	113 MB
API response, all 6 charts	62 KB
Pipeline peak memory	< 200 MB
VPS needed	2 GB RAM, 20 GB disk

Backups are deliberately minimal: the forecast is regenerated daily and there is no user data.

Future directions

- Localize plots to Finnish
- History browser (the array schema makes appending past runs easy)
- Allow users to subscribe to alerts via email
- Mobile app
 - Homescreen widget
 - Native notifications